

C++ Assignment 3

Question 1 — Network Request Dispatcher

Part A — enum class for HTTP Status Codes

```
#include <iostream>
using namespace std;

enum class HttpStatus {
    OK          = 200,
    Created      = 201,
    BadRequest   = 400,
    Unauthorized = 401,
    NotFound     = 404,
    ServerError  = 500
};

void handleResponse(HttpStatus status, const string& endpoint)
{
    cout << "[" << endpoint << "]" ";
    switch (status)
    {
        case HttpStatus::OK:
            cout << "→ 200 OK          : Request successful";
            break;

        case HttpStatus::Created:
            cout << "→ 201 Created      : Resource created successfully";
            break;

        case HttpStatus::BadRequest:
            cout << "→ 400 Bad Request   : Invalid request";
            break;

        case HttpStatus::Unauthorized:
            cout << "→ 401 Unauthorized  : Authentication required";
            break;

        case HttpStatus::NotFound:
            cout << "→ 404 Not Found     : Endpoint does not exist";
            break;
    }
}
```

```

        case HttpStatus::ServerError:
            cout << "→ 500 Server Error : Internal server error — retry later";
            break;
    }
    cout << endl;
}

int main() {
    handleResponse(HttpStatus::OK, "api/users");

    handleResponse(HttpStatus::Unauthorized, "api/login");

    handleResponse(HttpStatus::NotFound, "api/products/99");

    handleResponse(HttpStatus::ServerError, "api/order");

    return 0;
}

```

```

Problems Tasks Console × Include Browser Terminal Eclipse IDE for C/C++ Developers 2026-09 RC1
<terminated> (exit value: 0) Assignmentno3.exe [C/C++ Application] D:\CDAC\CPP\Assignmentno3\Debug\Assignmentno3.exe (9/2/26, 4:11 PM)
[/api/users] → 200 OK : Request successful
[/api/login] → 401 Unauthorized : Authentication required
[/api/products/99] → 404 Not Found : Endpoint does not exist
[/api/order] → 500 Server Error : Internal server error - retry later

```

Part B — Type Aliases for Readability

```

#include <iostream>
#include <string>
#include <vector>
#include <utility>

```

```

using namespace std;
using Header = pair<string, string>;
using HeaderList = vector<Header>;
using Port = unsigned int;
using IPAddress = string;

```

```

typedef unsigned long long RequestId;

```

```

void printHeaders(const HeaderList& headers)
{
    for (const Header& header : headers)

```

```

    {
        cout << header.first << "    : " << header.second << endl;
    }
}

int main()
{

    RequestId requestId = 1748293847;
    Port serverPort = 8080;
    HeaderList headers =
    {
        {"Content-Type", "application/json"},
        {"Authorization", "Bearer eyJhbGci..."},
        {"Accept-Language", "en-US"}
    };

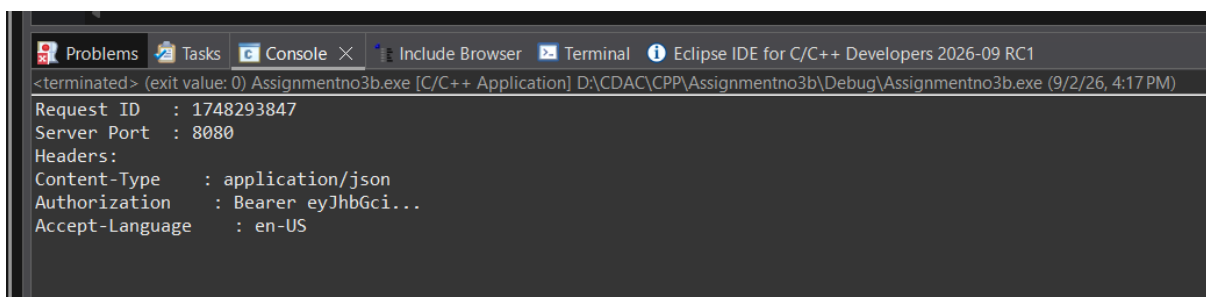
    cout << "Request ID   : " << requestId << endl;
    cout << "Server Port   : " << serverPort << endl;

    cout << "Headers:" << endl;

    // Print all headers
    printHeaders(headers);

    return 0;
}

```



The screenshot shows the Eclipse IDE interface with the 'Console' tab selected. The output text is as follows:

```

<terminated> (exit value: 0) Assignmentno3b.exe [C/C++ Application] D:\CDAC\CPP\Assignmentno3b\Debug\Assignmentno3b.exe (9/2/26, 4:17 PM)
Request ID   : 1748293847
Server Port   : 8080
Headers:
Content-Type   : application/json
Authorization   : Bearer eyJhbGci...
Accept-Language : en-US

```

Question 2 — Game Asset Manager

Part A — unique_ptr : Exclusive Asset Ownership

```
#include <iostream>
#include <string>
#include <memory>

using namespace std;

// Texture class
class Texture
{
private:
    string name;
    int width;
    int height;

public:
    // Constructor
    Texture(string n, int w, int h)
    {
        name = n;
        width = w;
        height = h;

        cout << "[Texture Loaded]" << endl;
    }

    // Destructor
    ~Texture()
    {
        cout << "[Texture Released]" << endl;
    }

    // Display function
    void display() const
    {
        cout << "Texture: " << name << endl;
        cout << "Dimensions: " << width << " x " << height << endl;
    }
};

int main()
```

```

{
    // Create Texture using unique_ptr
    auto tex1 = make_unique<Texture>("player_sprite", 512, 512);

    // Display texture
    tex1->display();

    // unique_ptr<Texture> tex2 = tex1;
    // Error: unique_ptr cannot be copied because it has exclusive ownership.

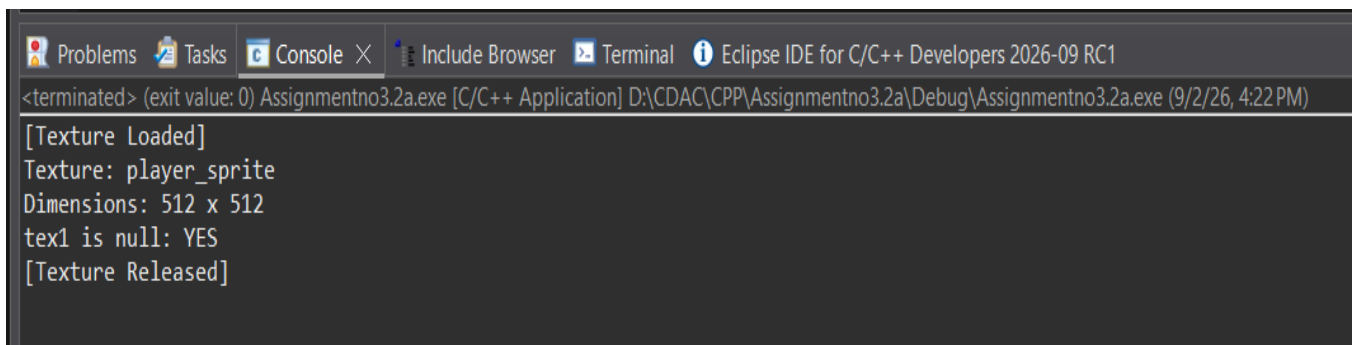
    // Transfer ownership from tex1 to tex2
    unique_ptr<Texture> tex2 = move(tex1);

    // Check whether tex1 is empty
    cout << "tex1 is null: "
        << (tex1 == nullptr ? "YES" : "NO") << endl;

    // tex2 goes out of scope at the end of main()
    // Destructor is called automatically.

    return 0;
}

```



The screenshot shows the Eclipse IDE interface with the 'Console' tab selected. The output text is as follows:

```

<terminated> (exit value: 0) Assignmentno3.2a.exe [C/C++ Application] D:\CDAC\CPP\Assignmentno3.2a\Debug\Assignmentno3.2a.exe (9/2/26, 4:22 PM)
[Texture Loaded]
Texture: player_sprite
Dimensions: 512 x 512
tex1 is null: YES
[Texture Released]

```

Part B — shared_ptr : Shared Shader Ownership

```
#include <iostream>
#include <string>
#include <memory>

using namespace std;
class Shader
{
private:
    string name;
    string type;

public:
    Shader(string n, string t)
    {
        name = n;
        type = t;

        cout << "[Shader Compiled]" << endl;
    }

    // Destructor
    ~Shader()
    {
        cout << "[Shader Destroyed]" << endl;
    }

    int getReferenceCount(shared_ptr<Shader> ptr)
    {
        return ptr.use_count();
    }
};

int main()
{
    auto shader = make_shared<Shader>("main_vert", "vertex");

    cout << "Ref count: " << shader.use_count() << endl;

    {
        auto rendererRef = shader;
```

```

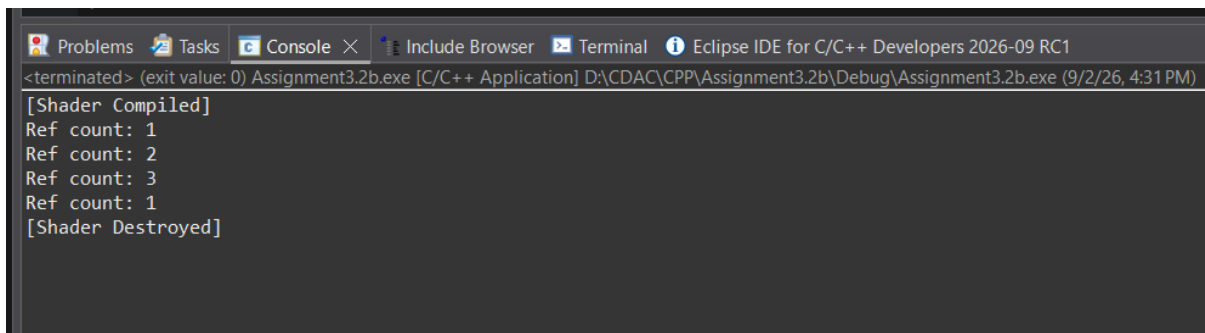
    cout << "Ref count: " << shader.use_count() << endl;

    auto editorRef = shader;

    cout << "Ref count: " << shader.use_count() << endl;
}
cout << "Ref count: " << shader.use_count() << endl;

return 0;
}

```



```

<terminated> (exit value: 0) Assignment3.2b.exe [C/C++ Application] D:\CDAC\CPP\Assignment3.2b\Debug\Assignment3.2b.exe (9/2/26, 4:31 PM)
[Shader Compiled]
Ref count: 1
Ref count: 2
Ref count: 3
Ref count: 1
[Shader Destroyed]

```

Part C — weak_ptr : Non-Owning Observer

```

#include <iostream>
#include <string>
#include <memory>

using namespace std;

class AudioClip
{
private:
    string name;
    double duration;

public:
    // Constructor
    AudioClip(string n, double d)
    {
        name = n;
        duration = d;

        cout << "[Audio Clip Loaded]" << endl;
    }
}

```

```

// Destructor
~AudioClip()
{
    cout << "[Audio Clip Released]" << endl;
}

// Function to get name
string getName() const
{
    return name;
}
};

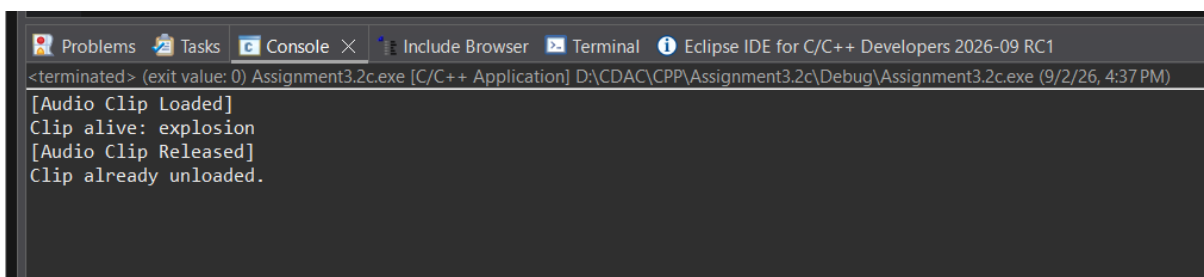
int main()
{

    auto audio = make_shared<AudioClip>("explosion", 3.5);
    weak_ptr<AudioClip> observer = audio;
    if (auto clip = observer.lock())
    {
        cout << "Clip alive: " << clip->getName() << endl;
    }
    audio.reset();

    if (observer.expired())
    {
        cout << "Clip already unloaded." << endl;
    }

    return 0;
}

```



The screenshot shows the Eclipse IDE interface with the 'Console' tab selected. The console output displays the following messages in sequence: '[Audio Clip Loaded]', 'Clip alive: explosion', '[Audio Clip Released]', and 'Clip already unloaded.'. The top of the window shows the Eclipse IDE title bar and menu bar, including 'Problems', 'Tasks', 'Console', 'Include Browser', 'Terminal', and 'Eclipse IDE for C/C++ Developers 2026-09 RC1'. The console title bar indicates the process is 'Assignment3.2c.exe' running at 'D:\CDAC\CPP\Assignment3.2c\Debug\Assignment3.2c.exe' on '9/2/26, 4:37 PM'.

```

<terminated> (exit value: 0) Assignment3.2c.exe [C/C++ Application] D:\CDAC\CPP\Assignment3.2c\Debug\Assignment3.2c.exe (9/2/26, 4:37 PM)
[Audio Clip Loaded]
Clip alive: explosion
[Audio Clip Released]
Clip already unloaded.

```

Question 3 — Log Buffer with Proper Copy Semantics

```
#include <iostream>
#include <cstring>
using namespace std;

class LogBuffer
{
private:
    char* buffer;
    int capacity;
    int size;

    static int instanceCount;

public:

    // Constructor
    LogBuffer(int capacity)
    {
        this->capacity = capacity;
        size = 0;

        buffer = new char[capacity + 1];
        buffer[0] = '\0';

        instanceCount++;

        cout << "[LogBuffer Created] capacity=" << capacity << endl;
    }

    // Copy Constructor
    LogBuffer(const LogBuffer& other)
    {
        capacity = other.capacity;
        size = other.size;

        buffer = new char[capacity + 1];

        strcpy(buffer, other.buffer);

        instanceCount++;

        cout << "[LogBuffer Deep Copied] capacity=" << capacity << endl;
```

```
}
```

```
// Copy Assignment Operator
```

```
LogBuffer& operator=(const LogBuffer& other)
```

```
{
```

```
    if (this == &other)
```

```
    {
```

```
        cout << "[Self-assignment detected — no operation]" << endl;
```

```
        return *this;
```

```
    }
```

```
    delete[] buffer;
```

```
    capacity = other.capacity;
```

```
    size = other.size;
```

```
    buffer = new char[capacity + 1];
```

```
    strcpy(buffer, other.buffer);
```

```
    cout << "[LogBuffer Assigned]" << endl;
```

```
    return *this;
```

```
}
```

```
// Destructor
```

```
~LogBuffer()
```

```
{
```

```
    delete[] buffer;
```

```
    instanceCount--;
```

```
    cout << "[LogBuffer Destroyed]" << endl;
```

```
}
```

```
// Append message
```

```
void append(const char* msg)
```

```
{
```

```
    strcat(buffer, msg);
```

```
    size = strlen(buffer);
```

```
}
```

```
// Print message
```

```
void print() const
```

```

    {
        cout << buffer << endl;
    }

    // Clear buffer
    void clear()
    {
        buffer[0] = '\0';
        size = 0;
    }

    // Get number of objects
    static int getInstanceCount()
    {
        return instanceCount;
    }
};

// Initialize static variable
int LogBuffer::instanceCount = 0;

int main()
{
    // Basic usage
    LogBuffer log1(256);

    log1.append("Server started on port 8080");
    log1.append(" | Request received from 192.168.1.10");

    log1.print();

    // Copy constructor
    LogBuffer log2 = log1;

    log2.append(" | Cached response sent");

    cout << "log1 : ";
    log1.print();

    cout << "log2 : ";
    log2.print();
}

```

```

// Copy assignment
LogBuffer log3(128);

log3 = log1;

cout << "log3 : ";
log3.print();

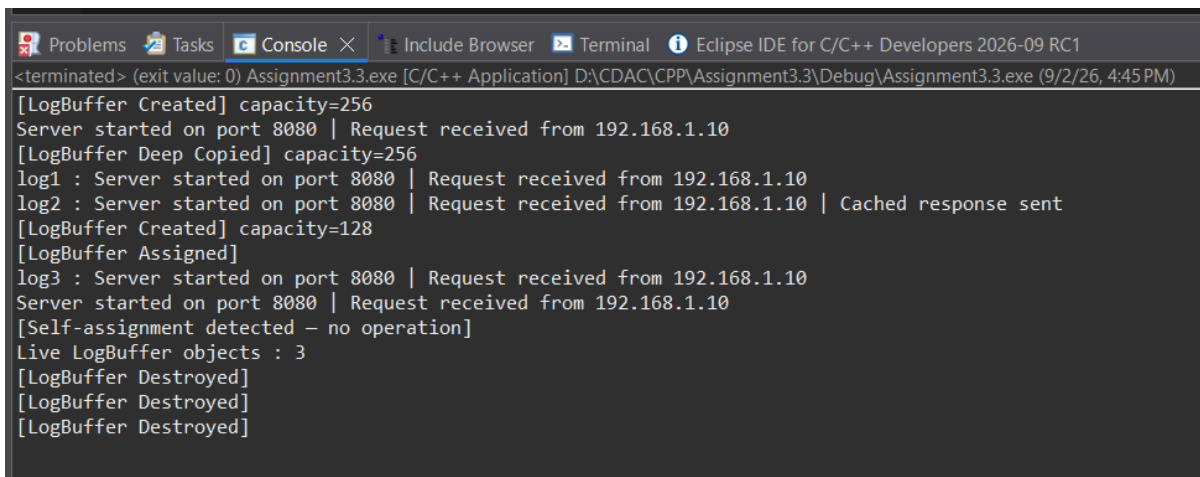
log1.print();

// Self-assignment
log1 = log1;

// Number of objects
cout << "Live LogBuffer objects : "
    << LogBuffer::getInstanceCount() << endl;

return 0;
}

```



The screenshot shows the Eclipse IDE interface with the 'Console' tab selected. The output text is as follows:

```

<terminated> (exit value: 0) Assignment3.3.exe [C/C++ Application] D:\CDAC\CPP\Assignment3.3\Debug\Assignment3.3.exe (9/2/26, 4:45 PM)
[LogBuffer Created] capacity=256
Server started on port 8080 | Request received from 192.168.1.10
[LogBuffer Deep Copied] capacity=256
log1 : Server started on port 8080 | Request received from 192.168.1.10
log2 : Server started on port 8080 | Request received from 192.168.1.10 | Cached response sent
[LogBuffer Created] capacity=128
[LogBuffer Assigned]
log3 : Server started on port 8080 | Request received from 192.168.1.10
Server started on port 8080 | Request received from 192.168.1.10
[Self-assignment detected - no operation]
Live LogBuffer objects : 3
[LogBuffer Destroyed]
[LogBuffer Destroyed]
[LogBuffer Destroyed]

```

Bonus — const Member Functions and mutable (Optional)

```
#include <iostream>
#include <cstring>
using namespace std;

class LogBuffer
{
private:
    char* buffer;
    int capacity;
    int size;

    static int instanceCount;

    // mutable allows this value to change inside a const function
    mutable int accessCount = 0;

public:

    // Constructor
    LogBuffer(int capacity)
    {
        this->capacity = capacity;
        size = 0;

        buffer = new char[capacity + 1];
        buffer[0] = '\0';

        instanceCount++;

        cout << "[LogBuffer Created] capacity=" << capacity << endl;
    }

    // Copy Constructor
    LogBuffer(const LogBuffer& other)
    {
        capacity = other.capacity;
        size = other.size;

        buffer = new char[capacity + 1];
        strcpy(buffer, other.buffer);
    }
};
```

```

    accessCount = 0;

    instanceCount++;

    cout << "[LogBuffer Deep Copied] capacity=" << capacity << endl;
}

// Copy Assignment Operator
LogBuffer& operator=(const LogBuffer& other)
{
    if (this == &other)
    {
        cout << "[Self-assignment detected — no operation]" << endl;
        return *this;
    }

    delete[] buffer;

    capacity = other.capacity;
    size = other.size;

    buffer = new char[capacity + 1];
    strcpy(buffer, other.buffer);

    accessCount = 0;

    cout << "[LogBuffer Assigned]" << endl;

    return *this;
}

// Destructor
~LogBuffer()
{
    delete[] buffer;
    instanceCount--;

    cout << "[LogBuffer Destroyed]" << endl;
}

// Append
void append(const char* msg)
{

```

```

        strcat(buffer, msg);
        size = strlen(buffer);
    }

    // const function
    void print() const
    {
        accessCount++;

        cout << buffer << endl;
    }

    // Getter
    int getAccessCount() const
    {
        return accessCount;
    }

    // Clear
    void clear()
    {
        buffer[0] = '\0';
        size = 0;
    }

    // Instance count
    static int getInstanceCount()
    {
        return instanceCount;
    }
};

int LogBuffer::instanceCount = 0;

int main()
{
    LogBuffer log1(256);

    log1.append("Server started on port 8080");
    log1.append(" | Request received from 192.168.1.10");

    log1.print();

```

```

LogBuffer log2 = log1;

log2.append(" | Cached response sent");

cout << "log1 : ";
log1.print();

cout << "log2 : ";
log2.print();

LogBuffer log3(128);

log3 = log1;

cout << "log3 : ";
log3.print();
log1 = log3;
const LogBuffer readOnly(log1);

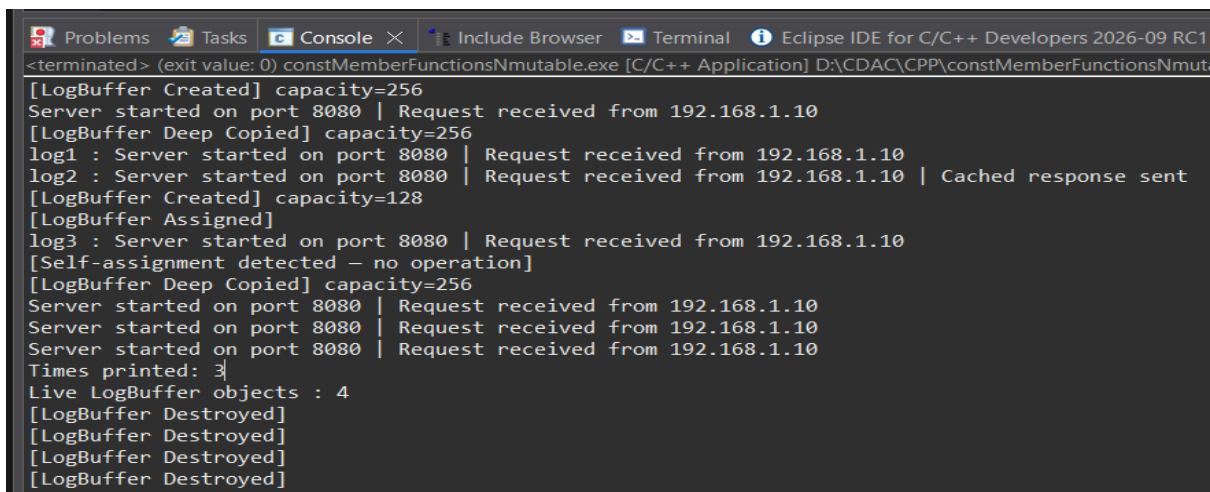
readOnly.print();
readOnly.print();
readOnly.print();

cout << "Times printed: "
    << readOnly.getAccessCount() << endl;

cout << "Live LogBuffer objects : "
    << LogBuffer::getInstanceCount() << endl;

return 0;
}

```



```

Eclipse IDE for C/C++ Developers 2026-09 RC1
<terminated> (exit value: 0) constMemberFunctionsNmutable.exe [C/C++ Application] D:\CDAC\CPP\constMemberFunctionsNmut
[LogBuffer Created] capacity=256
Server started on port 8080 | Request received from 192.168.1.10
[LogBuffer Deep Copied] capacity=256
log1 : Server started on port 8080 | Request received from 192.168.1.10
log2 : Server started on port 8080 | Request received from 192.168.1.10 | Cached response sent
[LogBuffer Created] capacity=128
[LogBuffer Assigned]
log3 : Server started on port 8080 | Request received from 192.168.1.10
[Self-assignment detected - no operation]
[LogBuffer Deep Copied] capacity=256
Server started on port 8080 | Request received from 192.168.1.10
Server started on port 8080 | Request received from 192.168.1.10
Server started on port 8080 | Request received from 192.168.1.10
Times printed: 3
Live LogBuffer objects : 4
[LogBuffer Destroyed]
[LogBuffer Destroyed]
[LogBuffer Destroyed]
[LogBuffer Destroyed]

```